

ZONE 4: REVIEW - CONTINUATION PART 2

Section 4.4: UI Components (Continued)

4.4.8: Best Practices Library Component

typescript

TypeScript

```
// components/review/BestPracticesLibrary.tsx
import React, { useState, useMemo } from 'react';
import { Card, CardHeader, CardTitle, CardContent } from
'@/components/ui/card';
import { Input } from '@/components/ui/input';
import { Badge } from '@/components/ui/badge';
import { Button } from '@/components/ui/button';
import { Tabs, TabsList, TabsTrigger, TabsContent } from
'@/components/ui/tabs';
import {
  Search,
  BookOpen,
  Star,
  TrendingUp,
  CheckCircle2,
  Download,
  Share2,
  Filter
} from 'lucide-react';

interface BestPractice {
  practice_id: string;
  category: 'resilience' | 'innovation' | 'coordination' |
'technology';
  title: string;
  description: string;
  source_incident_id?: string;
  effectiveness_score: number;
  adoption_rate: number;
```

```

cost_impact: 'low' | 'medium' | 'high';
implementation_difficulty: 'easy' | 'moderate' | 'complex';
tags: string[];
related_lessons: string[];
case_studies: Array<{
  venue_id: string;
  implementation_date: string;
  results: string;
  roi: number;
}>;
implementation_guide: {
  prerequisites: string[];
  steps: Array<{
    order: number;
    description: string;
    duration: string;
    resources_required: string[];
  }>;
  success_criteria: string[];
  common_pitfalls: string[];
};
created_at: string;
updated_at: string;
}

interface BestPracticesLibraryProps {
  event_id: string;
  venue_id: string;
  onApplyPractice: (practiceId: string) => Promise<void>;
}

const BestPracticesLibrary: React.FC<BestPracticesLibraryProps> =
({
  event_id,
  venue_id,
  onApplyPractice

```

```

    }) => {
      const [practices, setPractices] = useState<BestPractice[]>([]);
      const [searchTerm, setSearchTerm] = useState('');
      const [selectedCategory, setSelectedCategory] =
        useState<string>('all');
      const [selectedPractice, setSelectedPractice] =
        useState<BestPractice | null>(null);
      const [loading, setLoading] = useState(false);

      // Filter and search practices
      const filteredPractices = useMemo(() => {
        return practices.filter(practice => {
          const matchesSearch =
            practice.title.toLowerCase().includes(searchTerm.toLowerCase())
            ||
            practice.description.toLowerCase().includes(searchTerm.toLowerCase())
            ||
            practice.tags.some(tag =>
              tag.toLowerCase().includes(searchTerm.toLowerCase()));

          const matchesCategory = selectedCategory === 'all' ||
            practice.category === selectedCategory;

          return matchesSearch && matchesCategory;
        }).sort((a, b) => b.effectiveness_score -
          a.effectiveness_score);
      }, [practices, searchTerm, selectedCategory]);

      // Group practices by category
      const practicesByCategory = useMemo(() => {
        return filteredPractices.reduce((acc, practice) => {
          if (!acc[practice.category]) {
            acc[practice.category] = [];
          }
          acc[practice.category].push(practice);
        }, {});
      }, [filteredPractices]);
    }
  );
}

```

```

        return acc;
      }, {} as Record<string, BestPractice[]>);
    }, [filteredPractices]));

const getCostImpactColor = (impact: string) => {
  const colors = {
    low: 'bg-green-100 text-green-800',
    medium: 'bg-yellow-100 text-yellow-800',
    high: 'bg-red-100 text-red-800'
  };
  return colors[impact as keyof typeof colors] ||
colors.medium;
};

const getDifficultyColor = (difficulty: string) => {
  const colors = {
    easy: 'bg-green-100 text-green-800',
    moderate: 'bg-yellow-100 text-yellow-800',
    complex: 'bg-red-100 text-red-800'
  };
  return colors[difficulty as keyof typeof colors] ||
colors.moderate;
};

return (
  <div className="space-y-6">
    {/* Header & Search */}
    <Card className="border-l-4" style={{ borderLeftColor:
'#9AAF45' }}>
      <CardHeader>
        <div className="flex items-center justify-between">
          <div className="flex items-center gap-3">
            <div className="p-3 rounded-lg" style={{
backgroundColor: '#1B3A4B' }}>
              <BookOpen className="w-6 h-6 text-white" />
            </div>

```

```

        <div>
            <CardTitle className="text-2xl">Best Practices
Library</CardTitle>
            <p className="text-sm text-gray-600 mt-1">
                Proven strategies from across the venue network
            </p>
        </div>
    </div>
    <div className="flex gap-2">
        <Button variant="outline" size="sm">
            <Download className="w-4 h-4 mr-2" />
            Export Library
        </Button>
        <Button variant="outline" size="sm">
            <Share2 className="w-4 h-4 mr-2" />
            Share
        </Button>
    </div>
</div>
</CardHeader>
<CardContent>
    {/* Search and Filter */}
    <div className="flex gap-4 mb-6">
        <div className="flex-1 relative">
            <Search className="absolute left-3 top-1/2
transform -translate-y-1/2 text-gray-400 w-4 h-4" />
            <Input
                placeholder="Search best practices..."
                value={searchTerm}
                onChange={(e) => setSearchTerm(e.target.value)}
                className="pl-10"
            />
        </div>
        <Button variant="outline">
            <Filter className="w-4 h-4 mr-2" />
            Advanced Filters
        </Button>
    </div>

```

```

        </Button>
    </div>

    {/ * Category Tabs */}
    <Tabs value={selectedCategory}
onValueChange={setSelectedCategory}>
        <TabsList className="grid w-full grid-cols-5">
            <TabsTrigger value="all">All
({filteredPractices.length})</TabsTrigger>
            <TabsTrigger value="resilience">
                Resilience
({practicesByCategory.resilience?.length || 0})
            </TabsTrigger>
            <TabsTrigger value="innovation">
                Innovation
({practicesByCategory.innovation?.length || 0})
            </TabsTrigger>
            <TabsTrigger value="coordination">
                Coordination
({practicesByCategory.coordination?.length || 0})
            </TabsTrigger>
            <TabsTrigger value="technology">
                Technology
({practicesByCategory.technology?.length || 0})
            </TabsTrigger>
        </TabsList>

        <TabsContent value={selectedCategory}
className="mt-6">
            <div className="grid grid-cols-1 md:grid-cols-2
gap-4">
                {filteredPractices.map((practice) => (
                    <Card
                        key={practice.practice_id}
                        className="cursor-pointer hover:shadow-lg
transition-shadow"

```

```

        onClick={() => setSelectedPractice(practice)}
      >
      <CardContent className="p-6">
        {/* Practice Header */}
        <div className="flex items-start
justify-between mb-4">
          <div className="flex-1">
            <h3 className="font-semibold text-lg
mb-2">
              {practice.title}
            </h3>
            <p className="text-sm text-gray-600
line-clamp-2">
              {practice.description}
            </p>
          </div>
          <div className="flex items-center gap-1
ml-4">
            <Star className="w-5 h-5" style={{
color: '#9AAF45' }} fill="#9AAF45" />
            <span className="font-semibold">
              {practice.effectiveness_score.toFixed(1)}
            </span>
          </div>
        </div>

        {/* Metrics Row */}
        <div className="grid grid-cols-3 gap-4
mb-4">
          <div className="text-center">
            <div className="text-2xl font-bold"
style={{ color: '#3A7B8A' }}>
              {practice.adoption_rate}%
            </div>

```



```

        </Badge>
      </div>

      {/* Tags */}
      <div className="flex flex-wrap gap-2">
        {practice.tags.slice(0, 3).map((tag, idx)
=> (
          <Badge key={idx} variant="secondary"
className="text-xs">
            {tag}
          </Badge>
        ))}
        {practice.tags.length > 3 && (
          <Badge variant="secondary"
className="text-xs">
            +{practice.tags.length - 3} more
          </Badge>
        )}
      </div>
    </CardContent>
  </Card>
  )})
</div>
</TabsContent>
</Tabs>
</CardContent>
</Card>

{/* Detailed Practice View */}
{selectedPractice && (
  <Card>
    <CardHeader>
      <div className="flex items-start justify-between">
        <div>
          <CardTitle
className="text-2xl">{selectedPractice.title}</CardTitle>

```

```

                <p className="text-gray-600
mt-2">{selectedPractice.description}</p>
            </div>
            <Button
                onClick={() =>
onApplyPractice(selectedPractice.practice_id)}
                style={{ backgroundColor: '#9AAF45' }}
                className="text-white"
            >
                <CheckCircle2 className="w-4 h-4 mr-2" />
                Apply to Event
            </Button>
        </div>
    </CardHeader>
    <CardContent>
        <Tabs defaultValue="guide" className="w-full">
            <TabsList>
                <TabsTrigger value="guide">Implementation
Guide</TabsTrigger>
                <TabsTrigger value="cases">Case
Studies</TabsTrigger>
                <TabsTrigger value="metrics">Success
Metrics</TabsTrigger>
            </TabsList>

            <TabsContent value="guide" className="space-y-6">
                {/* Prerequisites */}
                <div>
                    <h4 className="font-semibold
mb-3">Prerequisites</h4>
                    <ul className="list-disc list-inside
space-y-2">

                        {selectedPractice.implementation_guide.prerequisites.map((prereq,
idx) => (

```

```

        <li key={idx}
className="text-gray-700">{prereq}</li>
      )}}
    </ul>
  </div>

  { /* Implementation Steps */ }
  <div>
    <h4 className="font-semibold
mb-3">Implementation Steps</h4>
    <div className="space-y-4">

      {selectedPractice.implementation_guide.steps.map((step) => (
        <div key={step.order} className="flex
gap-4">

          <div
            className="flex-shrink-0 w-8 h-8
rounded-full flex items-center justify-center text-white
font-semibold"
            style={{ backgroundColor: '#3A7B8A' }}
          >
            {step.order}
          </div>
          <div className="flex-1">
            <p className="font-medium
mb-1">{step.description}</p>
            <p className="text-sm
text-gray-600">Duration: {step.duration}</p>
            {step.resources_required.length > 0 &&
(
              <div className="mt-2">
                <p className="text-sm
font-medium">Resources Required:</p>
                <div className="flex flex-wrap
gap-2 mt-1">

```

```

{step.resources_required.map((resource, idx) => (
    <Badge key={idx}
    variant="outline">{resource}</Badge>
    ))}
</div>
</div>
)}
</div>
</div>
))}
</div>
</div>

{ /* Common Pitfalls */
<div>
    <h4 className="font-semibold mb-3">Common
Pitfalls to Avoid</h4>
    <div className="bg-red-50 border border-red-200
rounded-lg p-4">
        <ul className="list-disc list-inside
space-y-2">
{selectedPractice.implementation_guide.common_pitfalls.map((pitfall, idx) => (
            <li key={idx}
className="text-red-800">{pitfall}</li>
            ))}
        </ul>
    </div>
</div>
</TabsContent>

<TabsContent value="cases" className="space-y-4">
    {selectedPractice.case_studies.map((caseStudy,
idx) => (

```

```

        <Card key={idx}>
          <CardContent className="p-4">
            <div className="flex items-start
justify-between mb-3">
              <div>
                <h4 className="font-semibold">Venue
{caseStudy.venue_id}</h4>
                <p className="text-sm text-gray-600">
                  Implemented: {new
Date(caseStudy.implementation_date).toLocaleDateString()}
                </p>
              </div>
              <div className="text-right">
                <div className="text-2xl font-bold"
style={{ color: '#9AAF45' }}>
                  {caseStudy.roi}%
                </div>
                <div className="text-xs
text-gray-600">ROI</div>
              </div>
            </div>
            <p
className="text-gray-700">{caseStudy.results}</p>
          </CardContent>
        </Card>
      )})
    </TabsContent>

    <TabsContent value="metrics">
      <div className="space-y-4">
        <h4 className="font-semibold">Success
Criteria</h4>

        {selectedPractice.implementation_guide.success_criteria.map((crit
erion, idx) => (

```

```

                <div key={idx} className="flex items-start
gap-3">
                    <CheckCircle2 className="w-5 h-5 mt-0.5"
style={{ color: '#9AAF45' }} />
                    <p
className="text-gray-700">{criterion}</p>
                </div>
            )})
        </div>
    </TabsContent>
</Tabs>
</CardContent>
</Card>
    )}
</div>
);
};

export default BestPracticesLibrary;

```

4.4.9: Continuous Improvement Dashboard

typescript

```

TypeScript
// components/review/ContinuousImprovementDashboard.tsx
import React, { useState, useMemo } from 'react';
import { Card, CardHeader, CardTitle, CardContent } from
'@/components/ui/card';
import { Badge } from '@/components/ui/badge';
import { Button } from '@/components/ui/button';
import { Progress } from '@/components/ui/progress';
import {
    TrendingUp,
    Target,
    Zap,
    CheckCircle2,

```

```

    Clock,
    AlertCircle,
    ArrowUp,
    ArrowDown,
    Minus
} from 'lucide-react';
import {
    LineChart,
    Line,
    AreaChart,
    Area,
    BarChart,
    Bar,
    XAxis,
    YAxis,
    CartesianGrid,
    Tooltip,
    Legend,
    ResponsiveContainer,
    RadarChart,
    PolarGrid,
    PolarAngleAxis,
    PolarRadiusAxis,
    Radar
} from 'recharts';

interface ImprovementInitiative {
    initiative_id: string;
    title: string;
    category: 'resilience' | 'innovation' | 'coordination' |
'efficiency';
    status: 'planning' | 'in_progress' | 'completed' | 'on_hold';
    priority: 'low' | 'medium' | 'high' | 'critical';
    start_date: string;
    target_completion: string;
    actual_completion?: string;

```

```

    progress_percentage: number;
    metrics: {
      baseline: number;
      current: number;
      target: number;
      unit: string;
    };
    impact_score: number;
    roi_projection: number;
    dependencies: string[];
    owner: string;
    milestones: Array<{
      name: string;
      target_date: string;
      completed: boolean;
      completed_date?: string;
    }>;
  }

  interface MaturityScore {
    dimension: string;
    current_score: number;
    target_score: number;
    industry_average: number;
  }

  interface ContinuousImprovementDashboardProps {
    venue_id: string;
    time_range: { start: string; end: string };
  }

  const ContinuousImprovementDashboard:
  React.FC<ContinuousImprovementDashboardProps> = ({
    venue_id,
    time_range
  }) => {

```

```

    const [initiatives, setInitiatives] =
useState<ImprovementInitiative[]>([]);
    const [maturityScores, setMaturityScores] =
useState<MaturityScore[]>([]);
    const [selectedCategory, setSelectedCategory] =
useState<string>('all');

    // Calculate improvement velocity
    const improvementVelocity = useMemo(() => {
        const completed = initiatives.filter(i => i.status ===
'completed');
        const totalDays = completed.reduce((sum, i) => {
            if (i.actual_completion) {
                const start = new Date(i.start_date);
                const end = new Date(i.actual_completion);
                return sum + Math.floor((end.getTime() - start.getTime())
/ (1000 * 60 * 60 * 24));
            }
            return sum;
        }, 0);

        return completed.length > 0 ? Math.round(totalDays /
completed.length) : 0;
    }, [initiatives]);

    // Calculate aggregate metrics
    const aggregateMetrics = useMemo(() => {
        return {
            total_initiatives: initiatives.length,
            completed: initiatives.filter(i => i.status ===
'completed').length,
            in_progress: initiatives.filter(i => i.status ===
'in_progress').length,
            on_track: initiatives.filter(i =>
                i.progress_percentage >=

```

```

        (new Date().getTime() - new Date(i.start_date).getTime())
    /
        (new Date(i.target_completion).getTime() - new
Date(i.start_date).getTime()) * 100
    ).length,
    average_roi: initiatives.reduce((sum, i) => sum +
i.roi_projection, 0) / initiatives.length,
    average_impact: initiatives.reduce((sum, i) => sum +
i.impact_score, 0) / initiatives.length
    };
    }, [initiatives]);

const getStatusColor = (status: string) => {
    const colors = {
        planning: '#6B7280',
        in_progress: '#3A7B8A',
        completed: '#9AAF45',
        on_hold: '#EF4444'
    };
    return colors[status as keyof typeof colors] ||
colors.planning;
};

const getPriorityColor = (priority: string) => {
    const colors = {
        low: '#9AAF45',
        medium: '#3A7B8A',
        high: '#F59E0B',
        critical: '#EF4444'
    };
    return colors[priority as keyof typeof colors] ||
colors.medium;
};

const getTrendIcon = (current: number, baseline: number) => {

```

```

    if (current > baseline * 1.05) return <ArrowUp className="w-4
h-4 text-green-600" />;
    if (current < baseline * 0.95) return <ArrowDown
className="w-4 h-4 text-red-600" />;
    return <Minus className="w-4 h-4 text-gray-600" />;
  };

  return (
    <div className="space-y-6">
      { /* Header KPIs */ }
      <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
        <Card>
          <CardContent className="p-6">
            <div className="flex items-center justify-between">
              <div>
                <p className="text-sm text-gray-600">Active
Initiatives</p>
                <p className="text-3xl font-bold" style={{ color:
'#1B3A4B' }}>
                  {aggregateMetrics.in_progress}
                </p>
                <p className="text-xs text-gray-500 mt-1">
                  of {aggregateMetrics.total_initiatives} total
                </p>
              </div>
              <div className="p-3 rounded-lg" style={{
backgroundColor: '#1B3A4B' }}>
                <Zap className="w-6 h-6 text-white" />
              </div>
            </div>
          </CardContent>
        </Card>

        <Card>
          <CardContent className="p-6">
            <div className="flex items-center justify-between">

```

```

        <div>
            <p className="text-sm text-gray-600">Completion
Rate</p>
            <p className="text-3xl font-bold" style={{ color:
'#9AAF45' }}>
                {Math.round((aggregateMetrics.completed /
aggregateMetrics.total_initiatives) * 100)}%
            </p>
            <p className="text-xs text-gray-500 mt-1">
                {aggregateMetrics.completed} completed
            </p>
        </div>
        <div className="p-3 rounded-lg" style={{
backgroundColor: '#9AAF45' }}>
            <CheckCircle2 className="w-6 h-6 text-white" />
        </div>
    </div>
</CardContent>
</Card>

<Card>
    <CardContent className="p-6">
        <div className="flex items-center justify-between">
            <div>
                <p className="text-sm text-gray-600">Avg Impact
Score</p>
                <p className="text-3xl font-bold" style={{ color:
'#3A7B8A' }}>
                    {aggregateMetrics.average_impact.toFixed(1)}
                </p>
                <p className="text-xs text-gray-500 mt-1">
                    out of 10.0
                </p>
            </div>
            <div className="p-3 rounded-lg" style={{
backgroundColor: '#3A7B8A' }}>

```

```

        <Target className="w-6 h-6 text-white" />
      </div>
    </div>
  </CardContent>
</Card>

<Card>
  <CardContent className="p-6">
    <div className="flex items-center justify-between">
      <div>
        <p className="text-sm text-gray-600">Velocity</p>
        <p className="text-3xl font-bold" style={{ color:
'#1B3A4B' }}>
          {improvementVelocity}
        </p>
        <p className="text-xs text-gray-500 mt-1">
          days avg completion
        </p>
      </div>
      <div className="p-3 rounded-lg" style={{
backgroundColor: '#1B3A4B' }}>
        <TrendingUp className="w-6 h-6 text-white" />
      </div>
    </div>
  </CardContent>
</Card>
</div>

{/* Maturity Radar Chart */}
<Card>
  <CardHeader>
    <CardTitle>Operational Maturity Assessment</CardTitle>
  </CardHeader>
  <CardContent>
    <ResponsiveContainer width="100%" height={400}>
      <RadarChart data={maturityScores}>

```

```
<PolarGrid stroke="#E5E7EB" />
<PolarAngleAxis
  dataKey="dimension"
  tick={{ fill: '#6B7280', fontSize: 12 }}
/>
<PolarRadiusAxis
  angle={90}
  domain={[0, 100]}
  tick={{ fill: '#6B7280' }}
/>
<Radar
  name="Current Score"
  dataKey="current_score"
  stroke="#3A7B8A"
  fill="#3A7B8A"
  fillOpacity={0.5}
/>
<Radar
  name="Target Score"
  dataKey="target_score"
  stroke="#9AAF45"
  fill="#9AAF45"
  fillOpacity={0.3}
/>
<Radar
  name="Industry Average"
  dataKey="industry_average"
  stroke="#1B3A4B"
  fill="#1B3A4B"
  fillOpacity={0.1}
  strokeDasharray="5 5"
/>
<Legend />
<Tooltip />
</RadarChart>
</ResponsiveContainer>
```



```

        <Badge variant="outline">
            {initiative.category}
        </Badge>
    </div>
    <div className="flex items-center gap-4
text-sm text-gray-600">
        <span>Owner: {initiative.owner}</span>
        <span>•</span>
        <span>Target: {new
Date(initiative.target_completion).toLocaleDateString()}</span>
        <span>•</span>
        <span>ROI:
{initiative.roi_projection}%</span>
    </div>
</div>
<div className="text-right">
    <div className="text-2xl font-bold"
style={{ color: '#9AAF45' }}>
        {initiative.impact_score.toFixed(1)}
    </div>
    <div className="text-xs
text-gray-600">Impact Score</div>
</div>
</div>

    {/* Progress Bar */}
    <div className="mb-4">
        <div className="flex items-center
justify-between mb-2">
            <span className="text-sm
font-medium">Progress</span>
            <span className="text-sm
font-semibold">{initiative.progress_percentage}%</span>
        </div>
        <Progress
            value={initiative.progress_percentage}

```

```

        className="h-2"
      />
    </div>

    { /* Metrics */ }
    <div className="grid grid-cols-3 gap-4 mb-4">
      <div className="bg-gray-50 rounded-lg p-3">
        <div className="text-xs text-gray-600
mb-1">Baseline</div>
          <div className="flex items-center gap-2">
            <span className="text-lg font-semibold">
              {initiative.metrics.baseline}
            </span>
            <span className="text-xs
text-gray-500">{initiative.metrics.unit}</span>
          </div>
        </div>
        <div className="bg-gray-50 rounded-lg p-3">
          <div className="text-xs text-gray-600
mb-1">Current</div>
            <div className="flex items-center gap-2">
              <span className="text-lg font-semibold"
style={{ color: '#3A7B8A' }}>
                {initiative.metrics.current}
              </span>
              <span className="text-xs
text-gray-500">{initiative.metrics.unit}</span>
              {getTrendIcon(initialiative.metrics.current,
initiative.metrics.baseline)}
            </div>
          </div>
          <div className="bg-gray-50 rounded-lg p-3">
            <div className="text-xs text-gray-600
mb-1">Target</div>
              <div className="flex items-center gap-2">

```

```

                <span className="text-lg font-semibold"
style={{ color: '#9AAF45' }}>
                    {initiative.metrics.target}
                </span>
                <span className="text-xs
text-gray-500">{initiative.metrics.unit}</span>
            </div>
        </div>
    </div>

    { /* Milestones */ }
    <div>
        <div className="text-sm font-medium
mb-2">Milestones</div>
        <div className="space-y-2">
            {initiative.milestones.map((milestone, idx)
=> (
                <div key={idx} className="flex
items-center gap-3">
                    {milestone.completed ? (
                        <CheckCircle2 className="w-5 h-5"
style={{ color: '#9AAF45' }} />
                    ) : (
                        <Clock className="w-5 h-5
text-gray-400" />
                    )}
                    <span className={milestone.completed ?
'line-through text-gray-500' : ''}>
                        {milestone.name}
                    </span>
                    <span className="text-xs text-gray-500
ml-auto">
                        {new
Date(milestone.target_date).toLocaleDateString()}
                    </span>
                </div>
            )}
        </div>
    </div>

```

```

        ))}
      </div>
    </div>

    {/* Dependencies Warning */
    {initiative.dependencies.length > 0 && (
      <div className="mt-4 p-3 bg-yellow-50 border
border-yellow-200 rounded-lg flex items-start gap-2">
        <AlertCircle className="w-5 h-5
text-yellow-600 flex-shrink-0 mt-0.5" />
        <div>
          <p className="text-sm font-medium
text-yellow-800">
            Dependent on
{initiative.dependencies.length} other initiative(s)
          </p>
          <p className="text-xs text-yellow-700
mt-1">
            {initiative.dependencies.join(', ')}
          </p>
        </div>
      </div>
    )}
  </CardContent>
</Card>
  )}
</div>
</CardContent>
</Card>
</div>
);
};

export default ContinuousImprovementDashboard;

```